

Mathematical Theory of Multi-Layer Perceptron (supplemental material for MiniDXNN)

Takahiro Harada
Advanced Micro Devices, Inc.
USA

Sho Ikeda
Advanced Micro Devices, Inc.
Japan

ABSTRACT

This document provides a mathematical explanation of forward and backward propagation in Multi-Layer Perceptrons (MLPs), covering the fundamental equations and derivations used in training neural networks. We build intuition for each equation and explain the reasoning behind the mathematical formulations.

KEYWORDS

ray tracing, global illumination, octrees, quadrees

1 INTRODUCTION

A multi-layer perceptron (MLP) consists of fully connected neurons. Let L, n^l be the total number of layers, and the number of neurons in layer l . An operation on layer l can be represented with the weight matrix $\mathbf{W}^l \in \mathbb{R}^{n^l \times n^{l-1}}$ and the bias vector $\mathbf{b}^l \in \mathbb{R}^{n^l}$, followed by an activation. The weight matrix $\mathbf{W}^l$ encodes the strength of connection between layers, where W_{ij}^l represents the weight from neuron j in layer $l-1$ to neuron i in layer l . The bias vector $\mathbf{b}^l$ allows neurons to activate even when inputs are zero, effectively shifting the activation threshold.

The pre-activation $\mathbf{z}^l \in \mathbb{R}^{n^l}$ represents the raw output before applying the activation function, while the activation $\mathbf{a}^l \in \mathbb{R}^{n^l}$ is the actual output of neurons that gets passed to the next layer. We denote the activation function for layer l as σ^l .

In the following sections, we explain major operations we need for MLP, forward propagation, loss function, backpropagation, weight update, and activation one by one.

2 FORWARD PROPAGATION

Forward propagation computes the network's output given an input by successively applying linear transformations and activation functions. This is the **inference** or **prediction** phase of the neural network. Each layer receives an input, applies a linear transformation (weighted sum plus bias), and then applies a non-linear activation function. The output of one layer becomes the input to the next layer. This process repeats until we reach the output layer.

For each layer l (from 0 to $L-1$), the operation we execute is

$$\mathbf{z}^l = \mathbf{W}^l \mathbf{a}^{l-1} + \mathbf{b}^l \quad (1)$$

where $\mathbf{a}^{l-1}$ and $\mathbf{z}^l$ are the input and the output for layer l . The input of the MLP is when $l=0$, so $\mathbf{a}^0$ is the input to the MLP. The weights $\mathbf{W}^l$ defines how much each input influences each neuron, and the bias $\mathbf{b}^l$ shifts the value for activation, which we will explain later. Each equation in this vector form is $z_i^l = \sum_{j=1}^{n^{l-1}} W_{ij}^l a_j^{l-1} + b_i^l$.

After this computation, we apply activation:

$$\mathbf{a}^l = \sigma^l(\mathbf{z}^l) \quad (2)$$

Activation is an important step to introduce non-linearity to the system. Without activation functions, stacking multiple layers would be equivalent to a single linear transformation. The composition of linear functions is still linear: $f(g(x)) = \mathbf{W}^2(\mathbf{W}^1 x) = (\mathbf{W}^2 \mathbf{W}^1)x$. Non-linear activations allow the network to approximate arbitrarily complex functions. After processing through all layers, the final layer's activation represents the network's prediction $\hat{\mathbf{y}} = \mathbf{a}^{L-1}$.

Finally, here is the vectorized form of the operation at layer l .

$$\mathbf{Z}^l = \mathbf{W}^l \mathbf{A}^{l-1} + \mathbf{B}^l \quad (3)$$

$$\mathbf{A}^l = \sigma^l(\mathbf{Z}^l) \quad (4)$$

3 LOSS FUNCTIONS

The loss function $\mathcal{L}$ measures the discrepancy between predictions $\mathbf{y}$ and ground truth $\hat{\mathbf{y}}$. It quantifies how "wrong" the network's predictions are, providing a scalar value that we aim to minimize during training. Total loss over batch of m inputs is:

$$\mathcal{J} = \frac{1}{m} \sum_{j=1}^m \mathcal{L}(\hat{y}_j, y_j) \quad (5)$$

Averaging over the batch makes the loss independent of batch size, ensuring consistent gradient magnitudes across different batch sizes. There are many losses used in machine learning. Here, we show a few losses.

3.1 Mean Squared Error / L2 Loss

For regression tasks, mean square error (MSE) is often used. Let $\mathbf{y}, \hat{\mathbf{y}}$ be the grand truth (expected output) and the output from MLP. MSE loss can be written as

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \frac{1}{2} \|\hat{\mathbf{y}} - \mathbf{y}\|^2 = \frac{1}{2} \sum_{i=1}^n (\hat{y}_i - y_i)^2 \quad (6)$$

MSE penalizes the squared difference between predictions and targets. The factor of $\frac{1}{2}$ is a convenience that simplifies the derivative (it cancels with the power of 2). This loss function heavily penalizes large errors due to the squaring operation, is differentiable everywhere making optimization smooth.

In order to train the network, we need to compute the derivative of loss and use it to adjust the network parameters. As the loss is a function of $\hat{\mathbf{y}}$, derivative with respect to $\hat{\mathbf{y}}$ is:

$$\frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}} = \hat{\mathbf{y}} - \mathbf{y} \quad (7)$$

The gradient is simply the prediction error. If $\hat{y}_i > y_i$, the gradient is positive, pushing the prediction down through the back-propagation explained in the next section. If $\hat{y}_i < y_i$, the gradient is negative, pushing the prediction up.

3.2 Mean Absolute Error (MAE) / L1 Loss

L1 loss, also known as Mean Absolute Error (MAE), measures the absolute difference between predictions and targets:

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \|\hat{\mathbf{y}} - \mathbf{y}\|_1 = \sum_{i=1}^n |\hat{y}_i - y_i| \quad (8)$$

Unlike L2 loss, which squares the errors, L1 loss penalizes errors linearly. This makes L1 loss more robust to outliers because large errors contribute proportionally rather than quadratically to the total loss. The absolute value function treats all errors with equal weight per unit of error, while L2 loss increasingly emphasizes larger errors due to squaring.

Total loss over batch is

$$\mathcal{J} = \frac{1}{m} \sum_{j=1}^m \mathcal{L}(\hat{\mathbf{y}}_j, \mathbf{y}_j) \quad (9)$$

The derivative with respect to $\hat{\mathbf{y}}$ is:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}_i} = \begin{cases} 1 & \text{if } \hat{y}_i > y_i \\ -1 & \text{if } \hat{y}_i < y_i \\ \text{undefined} & \text{if } \hat{y}_i = y_i \end{cases} = \text{sign}(\hat{y}_i - y_i) \quad (10)$$

The gradient is the sign of the error. If the prediction is too high, the gradient is +1, pushing it down. If the prediction is too low, the gradient is -1, pushing it up. Unlike L2 loss where the gradient magnitude depends on the error size, L1 loss has constant gradient magnitude regardless of error size. This provides consistent updates but can make convergence less smooth near the optimum. In practice, the derivative at the exact point $\hat{y}_i = y_i$ is handled by using 0.

4 BACKWARD PROPAGATION (BACKPROPAGATION)

Backpropagation computes gradients of the loss function with respect to all parameters using the chain rule. It is called “backward” because we compute gradients starting from the output layer and propagating back toward the input layer. In a neural network, we need to know how changing any weight W_{ij}^l (potentially deep in the network) affects the final loss $\mathcal{L}$. Naively computing this by perturbing each weight individually would be possible if there are small number of parameters. However, it is impractical when the number of parameters is large, which is mostly the case for neural networks. Backpropagation computes all gradients in a single backward pass through the network in $O(W)$ time by cleverly reusing intermediate computations using the chain rule.

For a composite function $f(g(x))$, the chain rule states:

$$\frac{df}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx} \quad (11)$$

To see how f changes with x , we decompose it: see how f changes with g , then see how g changes with x , and multiply them together.

In neural networks, to compute $\frac{\partial \mathcal{L}}{\partial W_{ij}^l}$, we use the chain rule to decompose this into simpler pieces that we can compute efficiently.

The first step for backpropagation is the error δ^{L-1} at the output layer $l = L - 1$. It is computed as

$$\delta^{L-1} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{L-1}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{L-1}} \odot \frac{\partial \mathbf{a}^{L-1}}{\partial \mathbf{z}^{L-1}} \quad (12)$$

where $\odot$ denotes element-wise multiplication (Hadamard product). The term $\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{L-1}}$ tells us how much the loss changes with respect to the output activations, while $\frac{\partial \mathbf{a}^{L-1}}{\partial \mathbf{z}^{L-1}}$ tells us how much the activations change with respect to the pre-activations. The element-wise product combines these via the chain rule to get $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{L-1}}$.

For example, when we use MSE loss with identity activation, it can be computed as

$$\delta^{L-1} = \mathbf{a}^{L-1} - \mathbf{y} \quad (13)$$

Since $\sigma(z) = z$ (identity), we have $\frac{\partial \sigma}{\partial z} = 1$, and $\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{L-1}} = \mathbf{a}^{L-1} - \mathbf{y}$ for MSE.

The next step is computing the gradient for weights and biases. Using δ^{L-1} and chain rule, we can write them as

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{L-1}} = \delta^{L-1} (\mathbf{a}^{L-2})^T \quad (14)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{L-1}} = \delta^{L-1} \quad (15)$$

For weights, the gradient is the outer product of the error term δ^{L-1} and the previous layer’s activations $\mathbf{a}^{L-2}$. For biases, the gradient is simply the error term itself. Intuitive understanding of them are follows. A weight W_{ij}^{L-1} connects neuron j from layer $L - 2$ to neuron i in layer $L - 1$. The gradient should be proportional to both how wrong neuron i is (given by δ_i^{L-1}) and how active neuron j was (given by a_j^{L-2}). If $a_j^{L-2} = 0$, changing W_{ij}^{L-1} has no effect on the output, so the gradient should be zero. For biases, they affect the output regardless of previous activations, so the gradient only depends on the error.

To compute the gradient of parameters for hidden layer l , we first need to compute the error. The error is defined as $\delta_i^l = \frac{\partial \mathcal{L}}{\partial z_i^l}$. We apply the chain rule to consider all paths through which z_i^l affects $\mathcal{L}$. The loss depends on z_i^l only through its effect on $\mathbf{z}^{l+1}$:

$$\frac{\partial \mathcal{L}}{\partial z_i^l} = \sum_{j=1}^{n^{l+1}} \frac{\partial \mathcal{L}}{\partial z_j^{l+1}} \cdot \frac{\partial z_j^{l+1}}{\partial z_i^l} \quad (16)$$

From the forward pass equation $\mathbf{z}_j^{l+1} = \sum_{k=1}^{n^l} W_{jk}^{l+1} a_k^l + b_j^{l+1}$, taking the derivative with respect to z_i^l gives:

$$\frac{\partial z_j^{l+1}}{\partial z_i^l} = W_{ji}^{l+1} \cdot \frac{\partial a_i^l}{\partial z_i^l} \quad (17)$$

Only the term $k = i$ contributes (others don’t depend on z_i^l). Substituting back:

$$\delta_i^l = \sum_{j=1}^{n^{l+1}} \delta_j^{l+1} \cdot W_{ji}^{l+1} \cdot \frac{\partial a_i^l}{\partial z_i^l} \quad (18)$$

$$= \frac{\partial a_i^l}{\partial z_i^l} \sum_{j=1}^{n^{l+1}} W_{ji}^{l+1} \delta_j^{l+1} \quad (19)$$

In matrix form, the sum $\sum_{j=1}^{n^{l+1}} W_{ji}^{l+1} \delta_j^{l+1}$ is the i -th element of $(\mathbf{W}^{l+1})^T \boldsymbol{\delta}^{l+1}$, giving us the backpropagation equation as follows.

$$\boldsymbol{\delta}^l = \left((\mathbf{W}^{l+1})^T \boldsymbol{\delta}^{l+1} \right) \odot \frac{\partial \mathbf{a}^l}{\partial \mathbf{z}^l} \quad (20)$$

The first component, $(\mathbf{W}^{l+1})^T \boldsymbol{\delta}^{l+1}$, propagates the error from layer $l+1$ back to layer l through the weight matrix. The transpose is crucial as it reverses the direction of information flow. If neuron i in layer l strongly influences (through a large weight) a neuron in layer $l+1$ that has high error, then neuron i should receive a large error signal. The second component, $\frac{\partial \mathbf{a}^l}{\partial \mathbf{z}^l}$, scales by the derivative of the activation function. This accounts for how sensitive the activation is to changes in the pre-activation. For ReLU, $\frac{\partial a}{\partial z} = 0$ if $z < 0$, so no error flows through. For ReLU, $\frac{\partial a}{\partial z} = 1$ if $z > 0$ (active neuron), so the error is simply passed. The element-wise product $\odot$ combines these two factors for each neuron. The error at layer l is thus determined by how much each neuron contributed to errors in the next layer, and how sensitive each neuron's output is to changes in its input.

With error $\boldsymbol{\delta}^l$, we can now compute gradients for weights and biases. Using the chain rule, $\frac{\partial \mathcal{L}}{\partial W_{ij}^l} = \frac{\partial \mathcal{L}}{\partial z_i^l} \cdot \frac{\partial z_i^l}{\partial W_{ij}^l}$. From the forward pass $z_i^l = \sum_{k=1}^{n^{l-1}} W_{ik}^l a_k^{l-1} + b_i^l$, we get $\frac{\partial z_i^l}{\partial W_{ij}^l} = a_j^{l-1}$ (only the term $k = j$ contains W_{ij}^l). Therefore $\frac{\partial \mathcal{L}}{\partial W_{ij}^l} = \delta_i^l \cdot a_j^{l-1}$. In matrix form: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^l} = \boldsymbol{\delta}^l (\mathbf{a}^{l-1})^T$. Similarly, $\frac{\partial \mathcal{L}}{\partial b_i^l} = \frac{\partial \mathcal{L}}{\partial z_i^l} \cdot \frac{\partial z_i^l}{\partial b_i^l}$. Since $\frac{\partial z_i^l}{\partial b_i^l} = 1$, we get $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^l} = \boldsymbol{\delta}^l$. They can be written as follows in vector form.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^l} = \boldsymbol{\delta}^l (\mathbf{a}^{l-1})^T \quad (21)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^l} = \boldsymbol{\delta}^l \quad (22)$$

Note that they are the same form as the ones we used for the output layer.

The formulation we have been shown is for a single input. In reality, we pass many inputs in a batch. For a batch of m examples, we need to divide them by m .

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^l} = \frac{1}{m} \boldsymbol{\delta}^l (\mathbf{A}^{l-1})^T \quad (23)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^l} = \frac{1}{m} \sum_{j=1}^m \boldsymbol{\delta}_j^l \quad (24)$$

We take the mean gradient across all inputs. This provides a less noisy estimate of the true gradient compared to a single input. The sum is over all inputs in the batch (or equivalently, sum along the batch dimension).

5 TRAINING ALGORITHM SUMMARY

Here, we summarize the forward pass and backward pass. For the forward pass,

For $l = 1$ to $L - 1$:

$$\mathbf{z}^l = \mathbf{W}^l \mathbf{a}^{l-1} + \mathbf{b}^l \quad (25)$$

$$\mathbf{a}^l = \sigma^l(\mathbf{z}^l) \quad (26)$$

Store $\mathbf{z}^l$ and $\mathbf{a}^l$ for each layer. We will need these in the backward pass (or simply store $\mathbf{z}^l$ and apply activation before using $\mathbf{a}^l$).

The backward pass algorithm is as follows.

- (1) **Initialize:** Compute $\boldsymbol{\delta}^{L-1} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{L-1}} \odot \frac{\partial \mathbf{a}^{L-1}}{\partial \mathbf{z}^{L-1}}$
- (2) **Propagate:** For $l = L - 1$ down to 1:
 - Compute weight gradient: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^l} = \boldsymbol{\delta}^l (\mathbf{a}^{l-1})^T$
 - Compute bias gradient: $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^l} = \boldsymbol{\delta}^l$
 - If not at the first layer: Backpropagate error: $\boldsymbol{\delta}^{l-1} = \left((\mathbf{W}^l)^T \boldsymbol{\delta}^l \right) \odot \frac{\partial \mathbf{a}^{l-1}}{\partial \mathbf{z}^{l-1}}$

6 WEIGHT UPDATE RULES

Once gradients are computed, parameters are updated using optimization algorithms. The goal is to adjust weights and biases to reduce the loss.

6.1 Stochastic Gradient Descent (SGD)

Stochastic gradient descent is the most basic optimization algorithm:

$$\mathbf{W}^l := \mathbf{W}^l - \alpha \frac{\partial \mathcal{L}}{\partial \mathbf{W}^l} \quad (27)$$

$$\mathbf{b}^l := \mathbf{b}^l - \alpha \frac{\partial \mathcal{L}}{\partial \mathbf{b}^l} \quad (28)$$

Where α is the learning rate (a hyperparameter). In this algorithm, we move in the direction opposite to the gradient, which is the direction of steepest descent. This is because we want to decrease the loss.

6.2 Momentum

The momentum method adds a “velocity” term that accumulates past gradients.

$$\mathbf{v}_W^l := \beta \mathbf{v}_W^l + (1 - \beta) \frac{\partial \mathcal{L}}{\partial \mathbf{W}^l} \quad (29)$$

$$\mathbf{W}^l := \mathbf{W}^l - \alpha \mathbf{v}_W^l \quad (30)$$

where β is the momentum coefficient (typically 0.9). This is like a ball rolling down a hill. It accumulates velocity in consistent directions, dampens oscillations in directions where the gradient changes sign frequently, and can roll through small local minima due to inertia. This leads to faster convergence in ravines (elongated valleys), reduced oscillations, and the ability to escape shallow local minima. When $\beta = 0$, we have no momentum (standard SGD). With $\beta = 0.9$, the optimizer considers roughly the last 10 gradients. With $\beta = 0.99$, it considers roughly the last 100 gradients.

6.3 Adam (Adaptive Moment Estimation)

Adam combines momentum with adaptive learning rates:

$$\mathbf{m}_W^l := \beta_1 \mathbf{m}_W^l + (1 - \beta_1) \frac{\partial \mathcal{L}}{\partial \mathbf{W}^l} \quad (31)$$

$$\mathbf{v}_W^l := \beta_2 \mathbf{v}_W^l + (1 - \beta_2) \left(\frac{\partial \mathcal{L}}{\partial \mathbf{W}^l} \right)^2 \quad (32)$$

$$\hat{\mathbf{m}}_W^l := \frac{\mathbf{m}_W^l}{1 - \beta_1^t} \quad (33)$$

$$\hat{\mathbf{v}}_W^l := \frac{\mathbf{v}_W^l}{1 - \beta_2^t} \quad (34)$$

$$\mathbf{W}^l := \mathbf{W}^l - \alpha \frac{\hat{\mathbf{m}}_W^l}{\sqrt{\hat{\mathbf{v}}_W^l + \epsilon}} \quad (35)$$

The hyperparameters are $\beta_1 = 0.9$ (first moment decay, controlling the momentum), $\beta_2 = 0.999$ (second moment decay, controlling the adaptive learning rate), t is the iteration number, and $\epsilon = 10^{-8}$ provides numerical stability by preventing division by zero.

The first moment $\mathbf{m}_W^l$ is an exponential moving average of gradients (momentum). The second moment $\mathbf{v}_W^l$ is an exponential moving average of squared gradients, measuring variance or magnitude. The bias correction step (computing $\hat{\mathbf{m}}_W^l$ and $\hat{\mathbf{v}}_W^l$) is necessary because at early iterations, $\mathbf{m}_W^l$ and $\mathbf{v}_W^l$ are biased toward zero. Finally, the adaptive step gives each weight its own effective learning rate: $\frac{\alpha}{\sqrt{\hat{\mathbf{v}}_W^l + \epsilon}}$. Weights with consistently large gradients

get smaller effective learning rates (via large $\sqrt{\hat{\mathbf{v}}_W^l}$), while weights with small or infrequent gradients get larger effective learning rates. This adapts to the geometry of the loss landscape.

7 ACTIVATION FUNCTIONS AND THEIR DERIVATIVES

Activation functions introduce non-linearity. Their derivatives are crucial for backpropagation.

7.1 Identity (Linear)

$$\sigma(z) = z \quad (36)$$

$$\frac{d\sigma}{dz} = 1 \quad (37)$$

Identity activation can be used for the output layer for regression tasks where we want unbounded outputs.

7.2 Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (38)$$

$$\frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z)) \quad (39)$$

The sigmoid has output range $(0, 1)$, produces an S-shaped curve, and is smooth and differentiable everywhere. The derivative reaches its maximum at $z = 0$ where $\frac{d\sigma}{dz}|_{z=0} = 0.25$, and approaches zero for large $|z|$ (saturation). The sigmoid is used as the output layer for binary classification, or when you need outputs between 0 and 1.

7.3 Hyperbolic Tangent (tanh)

$$\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = \tanh(z) \quad (40)$$

$$\frac{d\sigma}{dz} = 1 - \sigma^2(z) \quad (41)$$

The tanh function has output range $(-1, 1)$, is zero-centered unlike sigmoid, and has maximum derivative $\frac{d\sigma}{dz}|_{z=0} = 1$.

7.4 ReLU (Rectified Linear Unit)

$$\sigma(z) = \max(0, z) \quad (42)$$

$$\frac{d\sigma}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases} \quad (43)$$

There's no vanishing gradient for $z > 0$. Also, it produces sparse activation, only a subset of neurons activate for any input, and it is computationally cheap.

7.5 Leaky ReLU

$$\sigma(z) = \max(\alpha z, z) \quad \text{where } \alpha \in (0, 1) \quad (44)$$

$$\frac{d\sigma}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ \alpha & \text{if } z \leq 0 \end{cases} \quad (45)$$

$\alpha = 0.01$ is often used. Leaky ReLU fixes the dying ReLU problem by allowing a small gradient even for negative values.

7.6 Softmax (for output layer)

For a vector $\mathbf{z} = [z_1, z_2, \dots, z_k]^T$:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}} \quad (46)$$

The softmax function ensures outputs sum to 1: $\sum_{i=1}^k \sigma(\mathbf{z})_i = 1$, all outputs are positive, and larger z_i values get exponentially more weight.

Derivative of softmax is

$$\frac{\partial \sigma_i}{\partial z_j} = \begin{cases} \sigma_i(1 - \sigma_i) & \text{if } i = j \\ -\sigma_i \sigma_j & \text{if } i \neq j \end{cases} \quad (47)$$

The diagonal terms ($i = j$) show how the i -th output changes with its own input, while the off-diagonal terms ($i \neq j$) show how the i -th output changes with other inputs (they compete due to normalization).

When softmax is combined with cross-entropy loss, it simplifies to

$$\frac{\partial \mathcal{L}}{\partial z_i} = \sigma_i - y_i \quad (48)$$

Despite the complex derivatives, when combined with cross-entropy loss, we get this simple form. This is one reason why softmax + cross-entropy is the standard for multi-class classification.

ACKNOWLEDGEMENT

We would like to thank Atsushi Yoshimura and Shin Fujieda for feedbacks.